AWS Cloud Practitioner

Student Lab Manual

Labs 1 – 8 · Region us-east-2 · prefix every resource with your username

Lab 1: Configuring IAM Roles and Policies

Duration: 30 minutes · **Region:** your assigned Region · **Prefix everything with your username:** `userN-`

🎯 Objectives

- Create an IAM group and user with a managed policy
- See least privilege in action by signing in as that user
- Create an IAM role for EC2 (used in Labs 3 and 5)

Exercise 1.1: Sign In (3 min)

1. Open the sign-in URL from your instructor and sign in with your IAM user name and password.
2. Top-right **Region selector** → choose **your assigned Region**. Every lab today runs there; check it whenever a console page looks empty.
3. Search for **IAM** and open it. Note the Region now shows **Global** — IAM is not Regional.

Exercise 1.2: Create a Group (4 min)

1. **User groups** → **Create group**.
2. Name: `userN-developers`.
3. Under permissions policies, check **`AmazonS3ReadOnlyAccess`**.
4. **Create user group**.

Exercise 1.3: Create a User (5 min)

1. **Users** → **Create user**. Name: `userN-dev`.
2. Check **Provide user access to the AWS Management Console** → *I want to create an IAM user*.
3. **Custom password**: pick one you'll remember. **Uncheck** *must create a new password at next sign-in*. **Next**.
4. **Add user to group** → check `userN-developers` → **Next** → **Create user**.
5. Copy the **Console sign-in URL** shown on the confirmation page.

Exercise 1.4: Test Least Privilege (13 min)

1. Open a **private/incognito window** and go to the sign-in URL. Sign in as `userN-dev`. Set the Region to your assigned Region.
2. Open **S3**. You can see the bucket list — the policy allows listing.
3. Choose **Create bucket**, type any name, and click **Create bucket** at the bottom. Read the error: `s3:CreateBucket` is not allowed.

4. Open **EC2** → **Instances**. You are denied — the policy says nothing about EC2, so it is implicitly denied.
5. Sign out and close the private window.

> IAM starts from "deny everything." `userN-dev` can do exactly what `AmazonS3ReadOnlyAccess` allows and nothing else.

Exercise 1.5: Create a Role for EC2 (5 min)

1. Back in your main window: **IAM** → **Roles** → **Create role**.
2. **Trusted entity:** AWS service. **Use case:** EC2. **Next**.
3. Check **`AmazonS3ReadOnlyAccess`**. **Next**.
4. **Role name:** `userN-ec2-role` → **Create role**.

In Lab 3 you attach this role to a server; in Lab 5 the server reads S3 with it — no access keys needed.

✅ Keep for later labs

`userN-developers`, `userN-dev`, `userN-ec2-role` — all deleted in Lab 8.

🧠 Knowledge Check

1. Why could `userN-dev` list buckets but not create one?
2. What is the difference between a user and a role?

Lab 2: Configuring Security Groups and Network ACLs

Duration: 30 minutes · **Region:** your assigned Region

🎯 Objectives

- Build a VPC with two public subnets, an Internet Gateway and a route table
- Create a web security group and a database security group
- Add a rule to a network ACL and compare it with a security group

What you build (used by Labs 3–7):

```
VPC userN-vpc 10.0.0.0/16
 ├─ Public subnet 1 (AZ a) ── web server (Lab 3)              security group: userN-
web-sg  (80, 22 from anywhere)
 ├─ Public subnet 2 (AZ b) ── web server 2 (Lab 4), RDS (Lab 7)   userN-db-sg (3306 from
userN-web-sg)
 └─ Route table: 0.0.0.0/0 → Internet Gateway
```

Exercise 2.1: Create the VPC (8 min)

1. Search **VPC** (confirm your assigned Region is selected) → **Create VPC** → choose **VPC and more**.
2. Settings:

Setting	Value
Name tag auto-generation	`userN`
IPv4 CIDR block	`10.0.0.0/16`
Number of Availability Zones	**2**
Number of public subnets	**2**
Number of private subnets	**0**
NAT gateways	**None**
VPC endpoints	None

3. **Create VPC**. Wait for the green checks, then **View VPC**.

Exercise 2.2: Enable Public IPs and Check the Route (5 min)

1. **Subnets** → for **each** of your two `userN-subnet-public…` subnets (the wizard named them after the two AZs it picked in your Region — call them **subnet 1** and **subnet 2** from here on): select it →

Actions → **Edit subnet settings** → check **Enable auto-assign public IPv4 address** → **Save**. (The wizard does not do this for you.)

2. **Route tables** → open `userN-rtb-public` → **Routes** tab. You will see `10.0.0.0/16 → local` and `0.0.0.0/0 → igw-…`. That second route is what makes the subnets public.

Exercise 2.3: Web Security Group (5 min)

1. **Security groups** → **Create security group**.
2. Name `userN-web-sg`, description `web server`, **VPC:** `userN-vpc` (not the default).
3. **Inbound rules** → add two:

Type	Source
HTTP	Anywhere-IPv4
SSH	Anywhere-IPv4

4. **Create security group**.

Exercise 2.4: Database Security Group (4 min)

1. **Create security group** again: name `userN-db-sg`, description `database`, VPC `userN-vpc`.
2. **Inbound rules** → add one: Type **MYSQL/Aurora**, Source: start typing `userN-web-sg` and select it.
3. **Create security group**.

> The database trusts *whatever server carries the web security group*, not an IP address. That is the standard AWS pattern.

Exercise 2.5: Network ACL (8 min)

1. **Network ACLs** → open the one whose VPC is `userN-vpc`. Look at **Inbound rules**: rule 100 allows all traffic; the last `*` rule denies everything else. This is the default NACL — wide open; the security groups do the real filtering.
2. **Edit inbound rules** → **Add new rule**:

Rule number	Type	Source	Allow/Deny
50	SSH (22)	`198.51.100.0/24`	**Deny**

3. **Save changes**. Rules are evaluated lowest number first, so rule 50 blocks SSH from that range before rule 100 allows everything.

> A security group can only *allow*; a network ACL can also *deny*, and it applies to every instance in the subnet.

✅ Checkpoint

- `userN-vpc` with 2 subnets, **auto-assign public IP enabled on both**
- `userN-web-sg` (80, 22) and `userN-db-sg` (3306 from web-sg)
- Default NACL has your Deny rule 50

🧠 Knowledge Check

1. Which route makes a subnet public?
2. Stateful or stateless: security group? network ACL?

Lab 3: Creating EC2 Instances

Duration: 25 minutes · **Region:** your assigned Region

🎯 Objectives

- Create a key pair and launch an Amazon Linux instance into your VPC
- Connect to it from the browser and confirm it is using its IAM role

---

Exercise 3.1: Key Pair (2 min)

1. **EC2** → **Key Pairs** → **Create key pair**.
2. Name `userN-key`, type RSA, format `.pem` → **Create key pair**. The file downloads; keep it.

Exercise 3.2: Launch the Instance (10 min)

Instances → **Launch instances**:

Setting	Value
Name	`userN-web-1`
AMI	**Amazon Linux 2023** (Quick Start)
Instance type	`t3.micro`
Key pair	`userN-key`
Network settings → **Edit** → VPC	`userN-vpc`
Subnet	`userN-subnet-public1-…` (subnet 1)
Auto-assign public IP	**Enable**
Firewall	**Select existing security group** → `userN-web-sg`
Advanced details → IAM instance profile	`userN-ec2-role`

Launch instance → **View all instances**. Wait for **Running** and **2/2 checks passed** (1–2 min).

Exercise 3.3: Look Around (4 min)

Select `userN-web-1` and check the tabs at the bottom:

- **Details:** Public IPv4 address, Private IPv4 address, Availability Zone, IAM Role
- **Security:** the rules from `userN-web-sg`
- **Storage:** the 8 GiB EBS root volume

Exercise 3.4: Connect (9 min)

1. **Connect** → **EC2 Instance Connect** tab → **Connect**. A terminal opens in the browser.
2. Run:

```
cat /etc/os-release | head -2      # which OS
ec2-metadata -i -z                  # instance ID and AZ
aws sts get-caller-identity         # who am I? → the userN-ec2-role
aws s3 ls                           # allowed by the role, no access keys anywhere
```

> If Connect fails: check the instance is Running, has a public IP, and `userN-web-sg` allows SSH. If `aws s3 ls` fails, the IAM role was not attached.

Leave the terminal open — Lab 4 continues here.

✅ Keep running

`userN-web-1` — used in Labs 4, 5 and 7.

🧠 Knowledge Check

1. Where did the credentials for `aws s3 ls` come from?
2. What did the AMI decide, and what did the instance type decide?

Lab 4: Deploying a Web Application on EC2

Duration: 30 minutes · **Region:** your assigned Region

🎯 Objectives

- Install a web server and deploy a page on your instance
- Create an AMI from it and launch a second identical server in another AZ

Exercise 4.1: Deploy on web-1 (10 min)

In the Instance Connect terminal for `userN-web-1`:

```
sudo dnf install -y httpd
sudo systemctl enable --now httpd
echo "<h1>Hello from $(ec2-metadata -i | cut -d' ' -f2) in $(ec2-metadata -z | cut -d' '
-f2)</h1>" | sudo tee /var/www/html/index.html
```

In the EC2 console copy the **Public IPv4 address** of `userN-web-1` and open `http://<public-ip>` in a new tab (http, not https). You should see the instance ID and the AZ of subnet 1.

> Page not loading? In order: instance Running → security group allows HTTP 80 → you typed `http://` → `sudo systemctl status httpd` shows active.

Exercise 4.2: Create an AMI (5 min)

1. Select `userN-web-1` → **Actions** → **Image and templates** → **Create image**.
2. Image name `userN-web-ami` → **Create image**.
3. **AMIs** (left menu) — it shows *Pending* for 2–4 minutes, then *Available*. Read ahead while you wait.

Exercise 4.3: Launch web-2 from the AMI (15 min)

The AMI has Apache and the page baked in, but the page still names web-1. A **user-data** script runs at first boot to fix that automatically.

1. **AMIs** → select `userN-web-ami` (Available) → **Launch instance from AMI**.
2. Same settings as Lab 3, except:

Setting	Value
Name	`userN-web-2`
Key pair	`userN-key`
VPC	`userN-vpc`
Subnet	**`userN-subnet-public2-…` (subnet 2 — the other AZ)**
Auto-assign public IP	Enable
Security group	existing → `userN-web-sg`
IAM instance profile	`userN-ec2-role`
Advanced details → **User data**	paste below

```
#!/bin/bash
echo "<h1>Hello from $(ec2-metadata -i | cut -d' ' -f2) in $(ec2-metadata -z | cut -d' '
-f2)</h1>" > /var/www/html/index.html
```

3. **Launch instance**. When Running, browse to **its** public IP. Different instance ID, **different AZ** — and you never logged in.

> Manual install → AMI → user data is how one server becomes a fleet. A load balancer in front of both is the next step (lecture).

✅ Keep running

`userN-web-1`, `userN-web-2`, `userN-web-ami` — cleaned up in Lab 8.

🧠 Knowledge Check

1. What does an AMI capture that user data does not?
2. Why put web-2 in a different Availability Zone?

Lab 5: Using S3 for Object Storage

Duration: 25 minutes · **Region:** your assigned Region

🎯 Objectives

- Create a bucket, upload an object, and recover an old version
- Host a static website from S3
- Read the bucket from EC2 using the IAM role (`userN-ec2-role`)

Before you start: download `lab05-index.html` and save it on your computer as **`index.html`**. (It is a one-page site that says "Version 1".)

Exercise 5.1: Create a Bucket (4 min)

1. Search **S3** → **Create bucket**.
2. Region: **your assigned Region**. Name `userN-bucket-XXXX` (XXXX = any four digits; names are globally unique).
3. **Bucket Versioning: Enable**. Leave everything else default. **Create bucket**.

Exercise 5.2: Upload and Version (6 min)

1. Open the bucket → **Upload** → **Add files** → `index.html` → **Upload** → **Close**.
2. Click `index.html` → **Open**. *Access Denied* — objects are private by default.
3. Open `index.html` in a text editor, change **Version 1** to **Version 2**, save, and upload it again.
4. In the Objects list, toggle **Show versions**. Two versions exist; the older one is still there and can be downloaded. That is how versioning protects against overwrites and deletes.

Exercise 5.3: Static Website (10 min)

1. **Properties** tab → **Static website hosting** → **Edit** → **Enable**. Index document `index.html` → **Save**. Note the **website endpoint** URL.
2. **Permissions** tab → **Block public access** → **Edit** → uncheck **Block all public access** → **Save** → confirm.
3. **Bucket policy** → **Edit** → paste (replace the bucket name) → **Save**:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::userN-bucket-XXXX/*"
```

```
  }]
}
```

4. Open the website endpoint in a new tab. Your page appears — no web server involved.

> Two locks had to open: Block Public Access and a bucket policy. That is deliberate — public buckets are the classic S3 mistake.

Exercise 5.4: Read from EC2 (5 min)

In the Instance Connect terminal for `userN-web-1` (reconnect if needed):

```
aws s3 ls s3://userN-bucket-XXXX/
aws s3 cp s3://userN-bucket-XXXX/index.html /tmp/ && cat /tmp/index.html
echo test > /tmp/t.txt && aws s3 cp /tmp/t.txt s3://userN-bucket-XXXX/
```

The last command fails: the role is **read-only**. Least privilege.

✅ Keep

`userN-bucket-XXXX` — deleted in Lab 8.

🧠 Knowledge Check

1. Why did the website return 403 before Exercise 5.3 step 3?
2. Which S3 feature let you get version 1 back?

Lab 6: Implementing Serverless Functions with Lambda

Duration: 25 minutes · **Region:** your assigned Region

🎯 Objectives

- Create and test a Python Lambda function
- Read its logs in CloudWatch
- Expose it over HTTPS with a Function URL

Exercise 6.1: Create the Function (4 min)

1. Search **Lambda** → **Create function** → **Author from scratch**.
2. Name `userN-hello`, Runtime **Python 3.13** (or newest). Leave the default new role. **Create function**.

Exercise 6.2: Code and Test (8 min)

1. In the **Code** tab, replace `lambda_function.py` with:

```
import json

def lambda_handler(event, context):
    print("Event received:", json.dumps(event))
    params = event.get("queryStringParameters") or {}
    name = params.get("name") or event.get("name", "World")
    return {
        "statusCode": 200,
        "body": f"Hello, {name}! Served by Lambda."
    }
```

2. **Deploy**.
3. **Test** → **Create new test event** → name `test1`, JSON `{ "name": "userN" }` → **Save** → **Test**.
4. Read the result panel: the returned body, **Duration**, **Billed duration** and **Max memory used**. That is what you pay for — and nothing while idle.

Exercise 6.3: Logs (5 min)

1. **Monitor** tab → graphs of invocations and duration.
2. **View CloudWatch logs** → open the newest log stream. Find your `Event received:` line between `START` and `END`.

Exercise 6.4: Function URL (8 min)

1. **Configuration** → **Function URL** → **Create function URL** → Auth type **NONE** → acknowledge → **Save**.
2. Copy the URL and open it in a browser. Add `?name=userN` to the end and reload.

> You just published an HTTPS endpoint with no server, no security group, no VPC. Compare with Labs 2–4.

✅ Keep

`userN-hello` — deleted in Lab 8.

🧠 Knowledge Check

1. What two things does Lambda bill for?
2. Give one job Lambda is *not* suited for.

Lab 7: Setting Up a Relational Database with RDS

Duration: Part A 5 min (before the break) + Part B 25 min · **Region:** your assigned Region

🎯 Objectives

- Launch a managed MySQL database inside your VPC
- Connect to it from your EC2 instance and create a table
- Find where backups and Multi-AZ live

> RDS takes 5–10 minutes to create. Do Part A now, take your break, come back to Part B.

Part A — Launch (5 min)

Exercise 7.1: DB Subnet Group (2 min)

1. Search **RDS** → **Subnet groups** → **Create DB subnet group**.
2. Name `userN-db-subnets`, description `lab`, VPC `userN-vpc`.
3. Availability Zones: select the **two AZs your subnets are in**. Subnets: select **both** of your subnets. **Create**.

Exercise 7.2: Create the Database (3 min)

Databases → **Create database** → **Standard create**. Everything not listed stays default:

Setting	Value
Engine	**MySQL**
Template	**Free tier** (or Dev/Test if not shown)
DB instance identifier	`userN-db`
Master username / password	`admin` / a password you write down
DB instance class	`db.t3.micro` (or db.t4g.micro)
Storage autoscaling	uncheck
Compute resource	Don't connect to an EC2 compute resource
VPC	`userN-vpc`
DB subnet group	`userN-db-subnets`
Public access	**No**
VPC security group	Choose existing → remove `default`, select **`userN-db-sg`**
Additional configuration → Initial database name	`labdb`
Deletion protection	uncheck

Create database. ☕ Take your break.

Part B — Use It (25 min)

Exercise 7.3: Endpoint (2 min)

Databases → `userN-db` (status **Available**) → **Connectivity & security** → copy the **Endpoint**.

Exercise 7.4: Connect from EC2 (13 min)

In the Instance Connect terminal for `userN-web-1`:

```
sudo dnf install -y mariadb105
mysql -h <your-endpoint> -u admin -p
```

Enter your password, then:

```
USE labdb;
CREATE TABLE visitors (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(50));
INSERT INTO visitors (name) VALUES ('userN'), ('Cloud Practitioner');
SELECT * FROM visitors;
EXIT;
```

> This worked only because `userN-web-1` carries `userN-web-sg`, which `userN-db-sg` trusts. Your laptop could not connect.

Exercise 7.5: Backups and High Availability (10 min)

On `userN-db` in the RDS console:

1. **Maintenance & backups** tab: automated backups with a retention period and a backup window — AWS runs these for you.
2. **Modify** (top right) → scroll to **Availability & durability**: *Multi-AZ* is a single option that adds a standby in another AZ with automatic failover. Scroll to **DB instance class**: this is how you scale. **Cancel** — do not apply.
3. **Monitoring** tab: CPU, connections and storage graphs from CloudWatch.

✅ Keep

`userN-db`, `userN-db-subnets` — deleted in Lab 8.

🧠 Knowledge Check

1. Who patches MySQL on `userN-db`? Who creates the tables?
2. Multi-AZ or read replica: which improves availability?

Lab 8: Implementing Cost Optimization Strategies

Duration: 30 minutes · **Region:** your assigned Region

🎯 Objectives

- Compare On-Demand and Savings Plan pricing for your web server
- Create a budget alert
- Rightsize an instance
- Clean up everything so nothing is billed overnight

Exercise 8.1: Price It (6 min)

1. Open https://calculator.aws → **Create estimate** → **Add service** → **Amazon EC2** → **Configure**.
2. Region: **your assigned Region**, Linux, instance **t3.micro**, quantity **2**, utilization 100%.
3. **Payment option: On-Demand** → note the monthly cost.
4. Change to **Compute Savings Plans, 1 year, No upfront** → note the new monthly cost.

> Same servers, ~30% cheaper for a one-year commitment. Commit for the steady baseline; stay On-Demand for spikes and experiments.

Exercise 8.2: Budget (6 min)

1. Search **Billing** → **Budgets** → **Create budget** → **Customize (advanced)** → **Cost budget** → **Next**.
2. Name `userN-budget`, Monthly, Fixed, amount **10** USD → **Next**.
3. **Add alert threshold**: 80%, **Actual**, your email → **Next** → **Next** → **Create budget**.

Exercise 8.3: Rightsize (6 min)

`userN-web-2` is oversized for a page that serves one line of HTML.

1. **EC2** → **Instances** → select `userN-web-2` → **Instance state** → **Stop instance**. Wait for *Stopped*.
2. **Actions** → **Instance settings** → **Change instance type** → **t3.nano** → **Apply**.
3. **Instance state** → **Start instance**. Browse to its (new) public IP — same page, half the price.

Exercise 8.4: Clean Up (12 min)

Work top to bottom.

☐	Resource	Where	How
☐	`userN-web-1`, `userN-web-2`	EC2 → Instances	Instance state → **Terminate**
☐	`userN-web-ami`	EC2 → AMIs	Actions → **Deregister AMI** → check *Delete associated snapshots*
☐	`userN-key`	EC2 → Key Pairs	Delete
☐	`userN-db`	RDS → Databases	Actions → **Delete** → uncheck *final snapshot* and *retain backups* → type `delete me`
☐	`userN-db-subnets`	RDS → Subnet groups	Delete (after the database is gone)
☐	`userN-hello`	Lambda	Actions → Delete
☐	`userN-bucket-XXXX`	S3	**Empty** first, then **Delete**
☐	`userN-vpc`	VPC → Your VPCs	Actions → **Delete VPC** (removes its subnets, IGW, `userN-web-sg` and `userN-db-sg`). If refused, the RDS instance is still deleting — wait and retry.
☐	`userN-budget`	Billing → Budgets	Delete
☐	`userN-dev`, `userN-developers`, `userN-ec2-role`	IAM	Delete each

Final check: EC2 Instances shows only *terminated* rows with your name; RDS and S3 show nothing of yours.

🧠 Knowledge Check

1. Which option would you pick for a server that runs 24×7 for three years? For a two-hour experiment?
2. What would still cost money if you had *stopped* the instances instead of terminating them?